

TECHNICAL RESEARCH REPORT
Comparative Study of Speech-to-Text Models
for Noisy Real-World Audio

Research, Benchmarking and Production Recommendation

Submitted by
Siddhesh Umesh Raut

Item	Details
Task	Research - Technical (AI/ML)
Models Evaluated	Whisper Base; Faster-Whisper Base; Wav2Vec2 Base-960h
Dataset	Mozilla Common Voice Hindi (100-utterance fixed evaluation subset)
Conditions	Clean speech and controlled 10 dB SNR noise
Primary Metrics	WER, inference latency, RTF, CPU memory usage

September 2026

Abstract

Automatic Speech Recognition (ASR) is a core component of voice-enabled applications, where recognition quality must be balanced with inference speed, memory consumption, and deployment constraints. This study compares three ASR configurations - OpenAI Whisper Base, Faster-Whisper Base, and Wav2Vec2 Base-960h - using a fixed 100-utterance Hindi subset from Mozilla Common Voice. Each model is evaluated under clean and controlled noisy conditions. The noisy condition is created by adding additive white Gaussian noise at 10 dB signal-to-noise ratio (SNR) while preserving the original reference transcript. The benchmark records Word Error Rate (WER), inference latency, real-time factor (RTF), and CPU memory usage. The study also examines model architecture, training data, licensing, hardware requirements, deployment characteristics, and suitability for noisy audio. The results show a clear trade-off: Wav2Vec2 provides the lowest measured latency and clean-condition WER on this specific subset but exhibits substantial degradation under noise and a fundamental English-vocabulary mismatch for Hindi references. Faster-Whisper provides sub-real-time CPU inference with lower memory usage than the standard PyTorch Whisper implementation and much smaller noise-induced WER degradation than Wav2Vec2. The final analysis therefore treats the benchmark as a scenario-specific engineering study rather than a universal ranking of ASR systems.

1. Introduction

Voice interfaces increasingly depend on ASR systems that can process speech reliably outside controlled laboratory conditions. Customer-support audio can contain environmental noise, microphone variation, accent differences, channel artifacts, and latency constraints. These factors make a single accuracy measure insufficient for production model selection.

The present work evaluates three complementary ASR approaches. Whisper is a multilingual sequence-to-sequence Transformer trained with large-scale weak supervision. Faster-Whisper retains the Whisper model but changes the inference runtime through CTranslate2. Wav2Vec2 uses a self-supervised acoustic representation followed by a CTC recognition head. The comparison is designed to separate model behavior from runtime engineering and to quantify the effect of a controlled noise stress test.

2. Problem Statement and Objectives

The practical problem is to identify an ASR configuration that provides an appropriate balance between recognition performance, latency, memory footprint, deployment simplicity, and robustness to acoustic noise for a voice-based customer-support workflow. The experiment uses the same evaluation utterances across the three systems so that the measurements are directly comparable within the chosen scenario.

- Benchmark three ASR implementations on a common fixed evaluation subset.
- Measure WER, latency, real-time factor, and CPU memory usage under clean and noisy conditions.
- Study model architecture, training data, licensing, and hardware/deployment requirements.
- Quantify noise-induced degradation using $\Delta \text{WER} = \text{WER}(\text{noisy}) - \text{WER}(\text{clean})$.
- Derive a scenario-specific recommendation and propose optimization, fine-tuning, and production strategies.

3. Model Research

3.1 OpenAI Whisper Base

Whisper is an automatic speech recognition and speech-translation model family developed by OpenAI. The released Base configuration contains approximately 74 million parameters and is available in both English-only and multilingual variants. The model is defined as a sequence-to-sequence Transformer and is designed to map audio to text using a unified encoder-decoder architecture [1,2].

Architecture. The released implementation first converts 16 kHz audio into an 80-channel log-Mel spectrogram. Two convolutional layers downsample the representation before a Transformer encoder processes the acoustic features. A six-layer Transformer decoder then generates output tokens autoregressively through causal self-attention and cross-attention. A multilingual BPE tokenizer and task/language prompt tokens allow the same architecture to support transcription and speech translation across multiple languages [1,2].

Training data. The Whisper paper reports 680,000 hours of weakly supervised audio and transcripts collected from the internet. The official model card gives a breakdown of approximately 438,000 hours of English audio with English transcripts, 126,000 hours of non-English audio paired with English transcripts, and 117,000 hours of non-English audio with corresponding transcripts; the non-English portion covers 98 languages [1,2].

Licensing and deployment. The official Whisper repository is MIT-licensed. The standard implementation is PyTorch-based and uses Ffmpeg for audio decoding. GPU execution can use half precision, while CPU execution falls back to full precision. The sequence-to-sequence decoding process introduces sequential compute during token generation, which can increase CPU latency [2,3].

Strengths and weaknesses. The official model card reports improved robustness to accents, background noise, and technical language compared with earlier systems, while also warning about hallucinations, repetitive text, and uneven performance across languages and accents [2]. This makes Whisper a strong general-purpose baseline, but domain-specific validation remains necessary before production deployment.

3.2 Faster-Whisper Base

Faster-Whisper is an optimized reimplementation of Whisper that uses the CTranslate2 inference engine. It is important to distinguish runtime optimization from model training: Faster-Whisper uses converted Whisper weights rather than defining a separately trained ASR architecture [4]. Consequently, the learned model remains Whisper Base while the execution engine changes.

Architecture and runtime. The underlying network retains Whisper's log-Mel front-end and encoder-decoder Transformer. CTranslate2 changes the execution path through a specialized C++ runtime, optimized memory handling, operator/kernel fusion, and support for multiple numerical precisions. The project also integrates PyAV for audio decoding and supports optional VAD-based speech segmentation [4,5].

Hardware and deployment. Faster-Whisper supports CPU and NVIDIA GPU execution and exposes quantization modes including INT8 and FP16 through the CTranslate2 runtime [4,5]. Its main engineering advantage is therefore not a different acoustic model, but a more efficient execution stack. In the present benchmark, the Base configuration was evaluated in CPU mode using INT8, providing a direct comparison with the standard PyTorch Whisper implementation.

3.3 Research Comparison at a Glance

Property	Whisper Base	Faster-Whisper Base
Learned model	Whisper Base, ~74M parameters	Same Whisper Base weights
Inference runtime	PyTorch	CTranslate2 / C++
Decoding	Autoregressive	Autoregressive, optimized runtime
Quantization	Standard release; CPU FP32	INT8, FP16 and related modes
License	MIT	MIT (project/runtime)
Primary advantage	Multilingual, broad robustness	Lower CPU latency and memory

3.4 Wav2Vec 2.0 Base-960h

Wav2Vec 2.0 is a self-supervised framework for learning speech representations from raw audio. The selected checkpoint, facebook/wav2vec2-base-960h, contains approximately 94.4 million parameters and combines a convolutional feature extractor, a Transformer context network, and a CTC output head [6,7].

Architecture. The model accepts mono audio sampled at 16 kHz. A seven-block temporal convolutional feature encoder transforms the waveform into latent acoustic representations. A 12-layer Transformer encoder then provides contextual representations, and a linear CTC layer predicts character emissions. Unlike Whisper, the selected Base-960h checkpoint has an English character vocabulary, so it is not a multilingual Hindi-capable checkpoint [6,7].

Training data and licensing. The checkpoint was pretrained and fine-tuned using 960 hours of LibriSpeech speech. The Wav2Vec 2.0 research introduced self-supervised pretraining with a contrastive objective over quantized latent representations, followed by supervised fine-tuning. The selected model checkpoint is distributed under the Apache 2.0 license [6,7].

Strengths and weaknesses. Wav2Vec2 has a non-autoregressive CTC decoding path and therefore can deliver very low inference latency. However, the selected checkpoint is English-specific and does not contain Devanagari characters. It also does not include the same broad weakly supervised web-speech diversity as Whisper, which limits direct transfer to the Hindi evaluation used in this study [6,7].

4. Dataset and Experimental Design

The benchmark uses a held-out Hindi partition from Mozilla Common Voice. The local project records 2,095 validated test utterances in the source split and selects 100 fixed utterances using a deterministic random seed of 42. The selected subset contains approximately 458.18 seconds of audio, with an average utterance duration of 4.58 seconds. Audio is loaded as mono and resampled to 16 kHz before inference. The reference transcripts remain in Devanagari Hindi.

Two conditions are evaluated. The clean condition uses the original Common Voice clips. The noisy condition uses the same 100 utterances after controlled additive white Gaussian noise is injected to achieve a 10 dB SNR. Because the references are preserved, the clean/noisy comparison isolates the effect of the synthetic acoustic perturbation within the limits of this test design.

5. Evaluation Methodology

All three systems use the same fixed evaluation subset and a batch size of one. One untimed warm-up inference is performed before latency measurement. Model download and first-time initialization are excluded from inference timing. The benchmark records prediction text, reference text, WER, audio duration, inference time, RTF, CPU RSS memory, and peak GPU allocation.

Metric	Definition / interpretation
WER	$WER = (S + D + I) / N$; lower values indicate fewer word-level recognition errors.
Inference latency	Elapsed time for transcription of one utterance, excluding model loading and warm-up.
RTF	Inference time divided by audio duration; $RTF < 1$ indicates faster-than-real-time processing.
CPU memory	Approximate resident set size measured with psutil.
GPU memory	Peak CUDA memory allocation; 0 MB when the benchmark executes on CPU.
Noise degradation	$\Delta WER = WER(\text{noisy}) - WER(\text{clean})$.

6. Experimental Results

The benchmark contains 100 clean and 100 noisy utterance conditions evaluated by each model. The reported values below are taken from the repository benchmark summary. Because the selected dataset is Hindi with Devanagari references while Wav2Vec2 Base-960h is an English-only checkpoint, the WER values should be interpreted as a cross-lingual stress test rather than as native-language accuracy for Wav2Vec2.

6.1 Benchmark Summary

Model	Clean WER	Noisy WER	Delta WER	Clean Latency	Noisy Latency
Faster-Whisper Base	1.0857	1.1057	+0.0200	2.336 s	3.502 s
Whisper Base	1.1018	1.1251	+0.0233	4.416 s	4.775 s
Wav2Vec2 Base-960h	1.0748	1.1847	+0.1099	0.376 s	0.368 s

6.2 Real-Time Factor and Resource Usage

Model	Clean RTF	Noisy RTF	CPU Memory	GPU Memory
Faster-Whisper Base	0.569	0.842	619.7 MB	0.0 MB
Whisper Base	1.149	1.131	840.1 MB	0.0 MB
Wav2Vec2 Base-960h	0.083	0.081	962.9 MB	0.0 MB

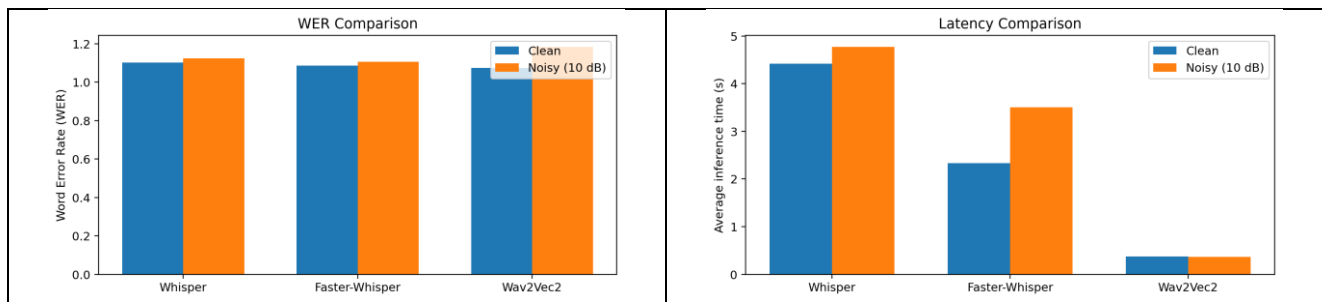


Figure 1. Clean versus noisy WER and inference-latency comparison from the measured benchmark.

The latency results show a strong runtime advantage for Wav2Vec2 on this CPU configuration, followed by Faster-Whisper and then the standard PyTorch Whisper implementation. However, latency alone does not determine suitability because the selected Wav2Vec2 checkpoint has an English-only output vocabulary and shows the largest noise-induced WER increase. Faster-Whisper retains Whisper's multilingual model while reducing CPU latency and memory consumption relative to the standard Whisper implementation. The latency comparison should be interpreted together with recognition quality and resource consumption. In the observed CPU-based experiment, Wav2Vec2 required the least inference time, indicating that the selected checkpoint can process the evaluation utterances with relatively low computational delay. This characteristic can be useful in applications where near-real-time transcription is important. However, lower latency does not automatically translate into better overall production suitability. The benchmark also evaluates recognition robustness, and the noisy condition introduced a larger deterioration in WER for Wav2Vec2 than for the Whisper-based configurations. Therefore, the latency advantage has to be considered against the model's behavior under degraded acoustic conditions. The Faster-Whisper implementation provides an important middle ground between recognition capability and runtime efficiency. Because it uses the Whisper model family with the CTranslate2 inference backend, it preserves the underlying Whisper model characteristics while improving practical inference efficiency. In the present CPU benchmark, Faster-Whisper required less inference time and lower CPU memory than the standard Whisper implementation. This difference is relevant for deployment environments where computational resources are constrained or where multiple transcription requests may need to be processed concurrently.

7. Comparative Analysis

The assessment requires comparison across accuracy, latency, resource usage, ease of deployment, and suitability for noisy audio. The matrix below combines measured benchmark evidence with documented model characteristics. It does not assign arbitrary scores; qualitative descriptions are tied to the observed measurements and the published architecture/runtime documentation.

Model	Recognition performance	Latency	Resource usage	Ease of deployment	Noisy-audio behavior
Faster-Whisper Base	Clean WER 1.0857; noisy WER 1.1057	2.336 s clean; 3.502 s noisy; RTF < 1	619.7 MB CPU; 0 MB GPU in CPU run	CTranslate2 runtime; CPU INT8; PyAV integration	Smallest Delta WER among the three (+0.0200)
Whisper Base	Clean WER 1.1018; noisy WER 1.1251	4.416 s clean; 4.775 s noisy; RTF about 1.1	840.1 MB CPU; 0 MB GPU in CPU run	Standard PyTorch stack; FFmpeg dependency	Small Delta WER (+0.0233); broad multilingual model
Wav2Vec2 Base-960h	Clean WER 1.0748; noisy WER 1.1847	0.376 s clean; 0.368 s noisy; RTF about 0.08	962.9 MB CPU; 0 MB GPU in CPU run	Transformers/PyTorch; strict 16 kHz input; English checkpoint	Largest Delta WER (+0.1099) and script/language mismatch

7.1 Interpretation of WER

All three measured WER values are close to or above 1.0, which is possible when insertions and substitutions are high. The dominant limitation is the test setup: the dataset contains Hindi speech with Devanagari ground truth, while Wav2Vec2 Base-960h has an English character vocabulary. Whisper is multilingual, but the base checkpoint can still emit alternative scripts or Romanized text under unprompted conditions. Therefore, these WER values should not be interpreted as a clean comparison of native Hindi ASR accuracy. Instead, they reveal how the selected off-the-shelf checkpoints behave under the exact data and language conditions used in the study.

7.2 Production-Oriented Recommendation

For the tested scenario - Hindi customer-support speech on CPU hardware with a requirement for practical real-time processing - Faster-Whisper is the most directly aligned configuration among the three evaluated options. The evidence is its sub-real-time clean and noisy RTF, lower CPU memory footprint than the standard Whisper implementation, multilingual Whisper model, and the smallest measured increase in WER under the 10 dB synthetic-noise condition. This is a scenario-specific engineering conclusion, not a universal claim that Faster-Whisper is the best ASR system for all languages or workloads.

The recommendation must still be validated with a Hindi-capable checkpoint/configuration, domain-specific call-center audio, and production hardware before deployment. The present benchmark intentionally exposes the limitations of using an English-only Wav2Vec2 checkpoint on Hindi and therefore should be considered a screening study rather than a production certification.

7.3 Optimization Strategy

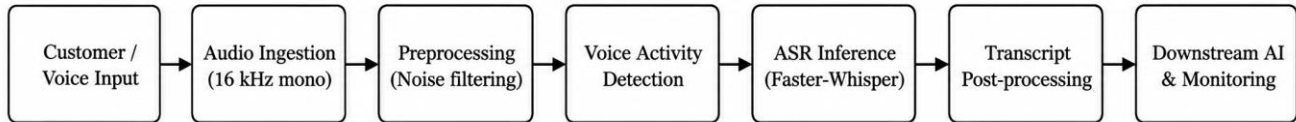
- Use INT8 quantization and the CTranslate2 runtime for CPU-serving efficiency where accuracy remains acceptable.
- Add voice activity detection to remove long non-speech segments before decoding and reduce unnecessary compute.
- Apply lightweight denoising or acoustic preprocessing only after validating that it improves WER on representative call-center recordings.
- Use dynamic batching or parallel workers for concurrent calls; keep single-stream batch size one for latency-sensitive interactions.

7.4 Fine-Tuning Strategy

A production adaptation study should collect representative telephony speech with accurate transcripts, customer-support terminology, speaker/accent diversity, and realistic channel artifacts. Split data at the speaker or call level to prevent leakage. Add telephony band-limiting, codec distortion, reverberation, and controlled noise during augmentation. Parameter-efficient fine-tuning may then be evaluated with a held-out call-centre test set. The same language, script, accent and acoustic conditions expected in deployment should be represented in the validation data.

8. Production Architecture

For a production voice assistant, the ASR component should be separated from audio ingestion and downstream conversational logic. The proposed architecture adds explicit preprocessing and monitoring so that audio quality, latency, and failure modes can be observed independently.



Production flow proposed from benchmark findings and deployment considerations

Figure 2. Proposed production architecture for an ASR-enabled customer-support voice assistant.

The pipeline begins with audio ingestion from SIP, WebRTC, or another supported source. Audio is normalized to a controlled sampling rate, followed by optional noise filtering and voice activity detection. The ASR service runs the selected model runtime and returns timestamps and text. Downstream post-processing may normalize phone numbers, order IDs, or other domain-specific entities. Monitoring should capture inference latency, audio duration, confidence-related signals where available, error counts, and service health without retaining unnecessary sensitive audio.

9. Limitations

- The evaluation uses a fixed subset of 100 utterances; the results should not be generalized to all Hindi speech or all customer-support traffic.
- The noisy condition is controlled 10 dB AWGN. It does not reproduce the full combination of babble, reverberation, telephony codec effects, packet loss, or overlapping speakers found in contact-center audio.
- The selected Wav2Vec2 checkpoint is English-only, so its Hindi results reflect a language/script mismatch as well as acoustic recognition behavior.
- The latency and memory measurements were obtained on a CPU runtime; GPU measurements can differ substantially.
- No domain-specific fine-tuning was performed in this benchmark; the fine-tuning strategy is therefore proposed future work.
- The benchmark is a comparative engineering study and should be followed by validation on representative production audio before deployment.

10. Conclusion

This study combines literature-based model research with a controlled benchmark of Whisper Base, Faster-Whisper Base, and Wav2Vec2 Base-960h. The measurements reveal three distinct engineering profiles: Whisper offers a broad multilingual baseline with higher CPU latency; Faster-Whisper retains the same model while improving runtime efficiency; and Wav2Vec2 provides the lowest latency but is poorly aligned with the Hindi evaluation because the selected checkpoint is English-only and more sensitive to noise.

Within the tested CPU-based Hindi scenario, Faster-Whisper provides the most practical balance of runtime efficiency, memory use, multilingual capability, and noise stability. This is limited to the benchmark conditions. Production validation should use a language-appropriate configuration, representative customer-support recordings, and a larger evaluation set.

11. References

1. Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I. (2022). Robust Speech Recognition via Large-Scale Weak Supervision. arXiv:2212.04356. <https://arxiv.org/abs/2212.04356>
2. OpenAI. Whisper Model Card. Official repository documentation. <https://github.com/openai/whisper/blob/main/model-card.md>
3. OpenAI. Whisper source repository and license. <https://github.com/openai/whisper>
4. SYSTRAN. Faster-Whisper: Faster Whisper transcription with CTranslate2. Official repository. <https://github.com/SYSTRAN/faster-whisper>
5. OpenNMT. CTranslate2 Documentation: Fast Inference Engine for Transformer Models. <https://opennmt.net/CTranslate2/>
6. Baevski, A., Zhou, H., Mohamed, A., and Auli, M. (2020). wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. NeurIPS 2020. <https://arxiv.org/abs/2006.11477>
7. Meta AI / Hugging Face. facebook/wav2vec2-base-960h model card and configuration. <https://huggingface.co/facebook/wav2vec2-base-960h>
8. Ardila, R., Branson, M., Davis, K., et al. (2020). Common Voice: A Massively-Multilingual Speech Corpus. LREC 2020. <https://aclanthology.org/2020.lrec-1.520/>
9. Graves, A., Fernandez, S., Gomez, F., and Schmidhuber, J. (2006). Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. ICML 2006. <https://doi.org/10.1145/1143844.1143891>

12. Repository and Experimental Record

The software implementation, benchmark tables, raw result summaries, research notes, and reproducibility material used for this report are maintained in the project repository: <https://github.com/Siddhesh511/asr-model-benchmark>

Report note: Published benchmark figures from the literature are cited for context only and are not substituted for the measured results in this study. The experimental values reported above are taken from the project benchmark summary.